

Operating Systems & Distributed Systems (CS301)

Authoritative Academic Curriculum & Lecture Notes

Chapter 1: Process Management & Lifecycle

A process is the fundamental unit of computation in an operating system, representing a program in execution. A process transitions through five primary lifecycle states: New, Ready, Running, Waiting (Blocked), and Terminated. The Operating System manages each active process via a Process Control Block (PCB), which encapsulates the Program Counter (PC), CPU registers, memory limits, and I/O status information. Context switching is the computational mechanism of saving the state of the currently executing process into its PCB and restoring the state of the next scheduled process.

Chapter 2: CPU Scheduling Algorithms (FCFS & SJF)

CPU scheduling decides which process in the ready queue is allocated the CPU core.

First-Come, First-Served (FCFS) is non-preemptive: tasks run strictly in arrival order. FCFS is vulnerable to the Convoy Effect, where short CPU bursts wait behind an intensive long burst, severely degrading average waiting time. Shortest-Job-First (SJF) selects the process with the minimum CPU burst time and is provably optimal for minimizing average waiting time. However, future burst durations cannot be known in advance, requiring exponential smoothing heuristics, and continuous short jobs can cause process starvation.

Chapter 3: Round-Robin Timesharing & Priority Scheduling

Round-Robin (RR) scheduling introduces preemption through a fixed time quantum (typically 10-100ms). The ready queue functions as a circular FIFO buffer. If the time quantum is excessively large, RR degrades into FCFS; if the quantum is overly brief, excessive context-switch overhead degrades CPU throughput. Priority scheduling assigns integer weights to processes, granting CPU cycles to the highest priority. A critical vulnerability is indefinite starvation of low-priority tasks. Operating systems prevent this via Aging, an adaptive technique that incrementally elevates a waiting process's priority over time.

Chapter 4: Synchronization, Mutexes, and Deadlocks

Concurrent execution requires mutual exclusion in critical sections to prevent race conditions. Semaphores provide integer-based signaling via atomic wait() and signal() operations. Deadlock occurs when concurrent processes are permanently blocked waiting for resources held by each other. Four Coffman conditions must hold simultaneously for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. The Banker's Algorithm by Edsger Dijkstra evaluates resource allocation safety to avoid deadlock.